Combining the Formal and the Informal

Jonathan Reicher Shachar Itzhaki

May 27, 2026

Problem Statement

In the intersection between the worlds of mathematical theory, formal logic, and programming, exists a species of tools called proof assistants. These are programming languages, meant not for building software, but for building mathematical proofs. Proof assistants, contrary to pen and paper, check formalized proofs for correctness and soundness.

These so called proof assistants admit only entirely formalized proofs, where every step is fully justified based on a complex logical semantics sitting on top of a complex declarative semantics. That is to say, not only are the logical steps themselves complex and rigorous, but so is the way you express your proof and how the steps are applied.

For many years, proof assistants where a niche, unpopular family of tools, used mostly by experts and enthusiasts. However, one member in particular has been gaining traction in the mathematical community, bringing many inexperienced users to the world of interactive theorem proving.

Lean(TODO: cite?) is an interactive proof assistant geared towards a general math audience. Two things that make it especially appealing are its large ecosystem and its intuitive syntax. For example, it has a large library of formalized mathematics called `mathlib`, which contains many foundational and advanced mathematical results and theories, and includes notation familiar to mathematicians. Despite that, there are some challenging aspects of adopting Lean, and one in particular is how complex its semantics and type system are.

We propose building a system that will reduce the friction of using Leanby being more "forgiving" and less formal. The system will allow users to write down proofs in way that humans can read and understand as written, with parts of the proof being translated to Lean, to help a user start formalizing their proof in Lean. This workflow is in accordance to how researchers tend to use Lean, as a formal proof in

LEANoften comes as a supplementary part of a paper proof inside a research paper.

Existing Solutions

For starting a new formal proof, one can reach for any off-the-shelf generative large language model such as ChatGPT. In an agentic setting, these systems have gotten good at generating idiomatic and correct LEANcode. One can write their paper, provide it to the agent, and prompt it to generate a formal LEANproof. There are even tools in place to help large language models work with LEANsuch Aristotle (TODO: cite?). These systems suffer a number of drawbacks. To name a few: They lack reproducibility, are usually based on cloud services, and are not fit with an interactive and iterative workflow.

There are also a number of automated proof machinery that can be embedded into LEAN, such as Aesop and Grind. These systems work on top of LEANby generating parts of a proof that you request. This is useful as part of interactive theorem proving, but they do not produce a readable proof. Arguably, they can start from an informal proof, if the statement in question is formalized as a theorem signature in LEAN, with a missing or a partial proof body.

TODO: Mention open ai's paper that adi sent

TODO: Mention LeanEgg

Our Approach

We define a language for writing informal proofs, with the formal parts being recognized and translated to matching LEANcode.

express parts of the proof in formal

Preliminary Work

As a case study, we started by taking a look at the following problem:

> Can you cover a 10×10 board with 1×2 tiles, such that the number of horizontal and vertical tiles is the same?

As it turns out, you can't. One can prove that it is impossible. A short intuitive proof can be seen in Figure 1. While this proof is

Proposition	Explanation
$\lvert D\rvert = 50$	Our board's size is 100, and every element is a set of size 2, hence: 50 tiles
$(D \cap H) \cup (D \cap V) = D$	By $D \subseteq H \cup V$
$(D \cap H) \cap (D \cap V) = \varnothing$	Because H and V are disjoint!
$\lvert D \cap H\rvert = \lvert D \cap V\rvert = 25$	By the assumption, and by previous two conjectures
$\sum\{\!\{x \mid \langle x, y\rangle \in^2 D \cap V\}\!\}$ is even	Every number in the sum appears twice, as the x's of a tile's cells are equal!
$\sum\{\!\{x \mid \langle x, y\rangle \in^2 D \cap H\}\!\}$ is odd	Here every number appears with its successor by similar reasoning
$\sum\{\!\{x \mid \langle x, y\rangle \in 0..9 \times 0..9\}\!\}$ is even	By decide
$\sum\{\!\{x \mid \langle x, y\rangle \in 0..9 \times 0..9\}\!\} = \sum\{\!\{x \mid \langle x, y\rangle \in^2 D \cap V\}\!\} + \sum\{\!\{x \mid \langle x, y\rangle \in^2 D \cap H\}\!\}$	todo
Contradiction	todo

Figure 1: A proof sketch of the problem.

readable and correct, and even relatively formal, it does not translate well to Lean.

As an example, here is a part of the proof that represents conjecture ???.

One of the challenges in translating a proof to Leanis representation. It is often much more idiomatic to use one of many possible representations. For something that might fit as a set for a mathematician, in Leanit might be better use the `Set` type, or the `Finset`, or maybe represent it as a type, or even a predicate. The most idiomatic choice for representation depends on use.

As a proof of concept, we made a tool which compiles inline math in a markdown document to Leancode, and uses a static analysis of the document to determine which representation to use for each mathematical object. You can invoke the tool interactively inside a Leandocument, giving it a markdown document, and it declares variables and definitions which can be used from that point on as if they were declared in the file itself directly.

The tool we have right now is very basic, and the only possible representations are `Set` or `Finset` for sets, and `Multiset` for multisets and `Prod` for tuples.

Example For example, for the following markdown document:

```
Let $H = \{ \{ (x, y), (x + 1, y) \} \mid x \in 0..8, y \in
0..9 \} \}$ be the set of horizontal tiles on the board.
```

The tool generates the following LEANcode:

```
 def H : Set (Fin 10 × Fin 10) := { (x, y) | x ∈ Finset.range 9, y ∈ Finset.range
10 }
```

The tool is invoked by simply calling the macro `define_latex "foo.md"`, with the file name of the markdown document as the argument.

References